

Hardware303

Christian Miranda

I have implemented a partial replica of the Roland TB-303 in VHDL. The Roland TB-303 is an analog bass line synthesizer noted for its characteristic filter - a diode ladder filter - that is hard to replicate in software. This design uses a Moog ladder filter however, since it is much more well known, and does not require significant analog analysis to replicate the sound of the TB-303 as a diode ladder would.

Theory of Operation

Basic Blocks

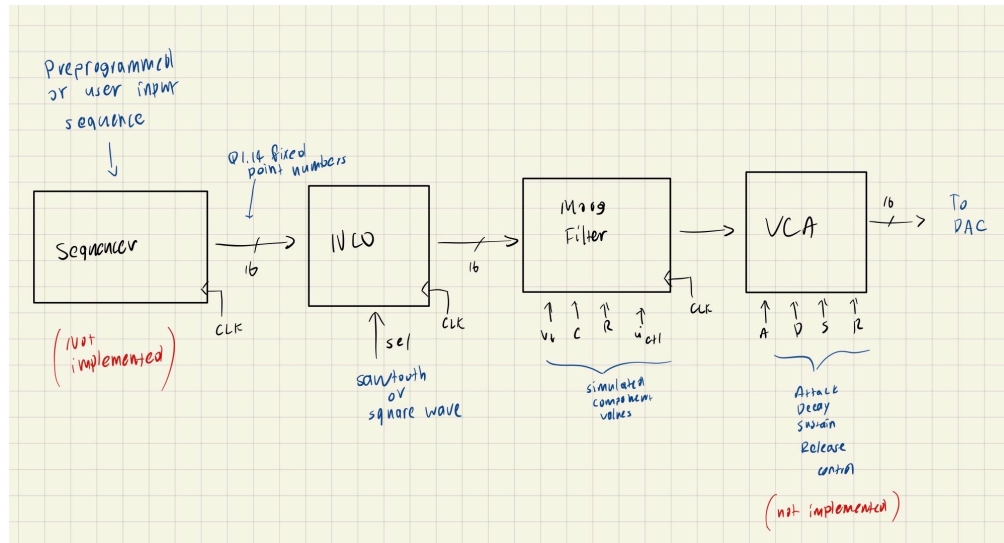


Figure 1: Hardware 303 Block diagram

The Roland TB-303 consists of the following blocks: - Oscillator: a voltage-controlled oscillator that creates a square or sawtooth wave in the bass clef range. - Filter: a resonant low-pass filter meant to mimic the sound of a Moog ladder filter, but using diodes instead of BJTs for the active component of the filter. - Sequencer: a small bank of memory and logic that generates the frequencies to be produced by the oscillator. - VCA: a voltage controlled amplifier used to control the dynamics (volume before, during, and after a note is played). The TB-303 has the controls of ADSR (attack, decay, sustain, and release).

Thus, the Hardware 303 should implement the digital equivalent of each of these blocks. That is,

- A numerically controlled oscillator that takes an integer and produces both a square and sawtooth wave of the desired frequency.
- A Moog ladder filter that models the dynamics of the analog filter.
- A sequencer that generates integers according to a user input pattern.
- A NCA (numerically controlled amplifier) that prescales the output according to the desired setting.

In this project, an NCO and filter were implemented and tested in VHDL.

NCO

An NCO capable of outputting a square and sawtooth wave essentially consists of an adder, a comparator, and a few registers to accumulate the added value. The input word is added each clock period, and if a

sawtooth waveform is desired, we just output the accumulated value. If a square wave output is desired, we simply threshold the accumulated value according to our desired duty cycle. More complex NCOs included phase control word inputs in addition to frequency control inputs, allowing the user to shift the phase when desired. If a sine wave is desired, a lookup table mapping an accumulated value to an output value can be implemented.

The NCO was tested over the bass clef range by logging the output word at each clock cycle to a text file. This text file was then analyzed with a Python script, `verify_nco.py`. This generates both a time and frequency domain plot for each output frequency. We also programmitically verify that besides just the fundamental frequency being correct, the spectral contents of the waveform are what we expect for a square and sawtooth wave. A perfect square wave will contain only odd harmonics, while a sawtooth wave will include all harmonics. The harmonics of both will decay at a rate of 6dB / octave.

The NCO was implemented in the `nco` directory, specifically in `nco.vhd` and `VHD`, and a test bench `nco_tb.vhd` as well as an accompanying python script rigorously tested it's functionality over our desired frequency range.

Filter

A resonant low-pass filter is simply a low-pass filter that produces The filter is based on the Huovilainen's paper titled "NON-LINEAR DIGITAL IMPLEMENTATION OF THE MOOG LADDER FILTER" [1], in which the author aproximates the non-linearities of a Moog ladder filter, resulting in the following input-output equations:

$$y_a(n) = y_a(n-1) + \frac{I_{ctl}}{CF_s} \left(\tanh \left(\frac{x(n) - 4ry_d(n-1)}{2V_t} \right) - W_a(n-1) \right)$$

$$y_b(n) = y_b(n-1) + \frac{I_{ctl}}{CF_s} (W_a(n) - W_b(n-1))$$

$$y_c(n) = y_c(n-1) + \frac{I_{ctl}}{CF_s} (W_b(n) - W_c(n-1))$$

$$y_d(n) = y_d(n-1) + \frac{I_{ctl}}{CF_s} \left(W_c(n) - \tanh \left(\frac{y_d(n-1)}{2V_t} \right) \right)$$

$$W_{\{a,b,c\}}(n) = \tanh \left(\frac{y_{\{a,b,c\}}(n)}{2V_t} \right)$$

where $x[n]$ is the output of NCO, and y_d is the output of the filter. We naturally see that this design lends itself to a multistage design, in which we can generically define a filter stage, as well as a generic "W block" that computes $W_{\{a,b,c\}}(n)$.

The W block can be represented as follows:

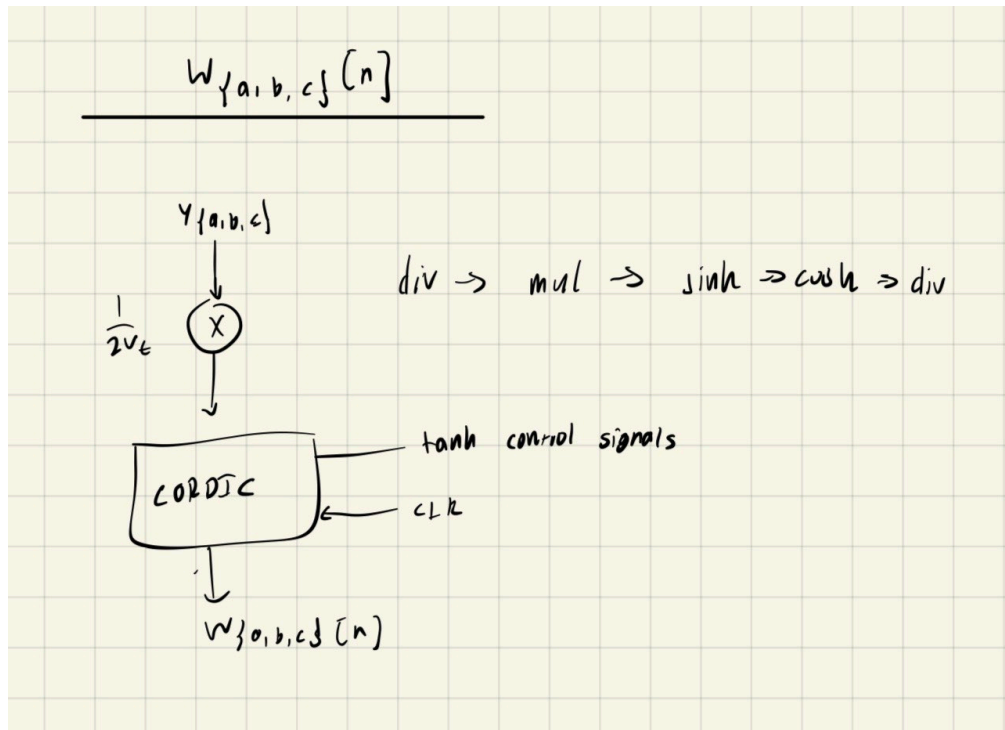


Figure 2: W block diagram

Pratically, we implement division and mulitplication with the CORDIC as well. Additionally, since the CORDIC can only calculate sinh, and cosh, we need a final division step to calculate tanh. This can be done with the following arrangement of CORDICs:

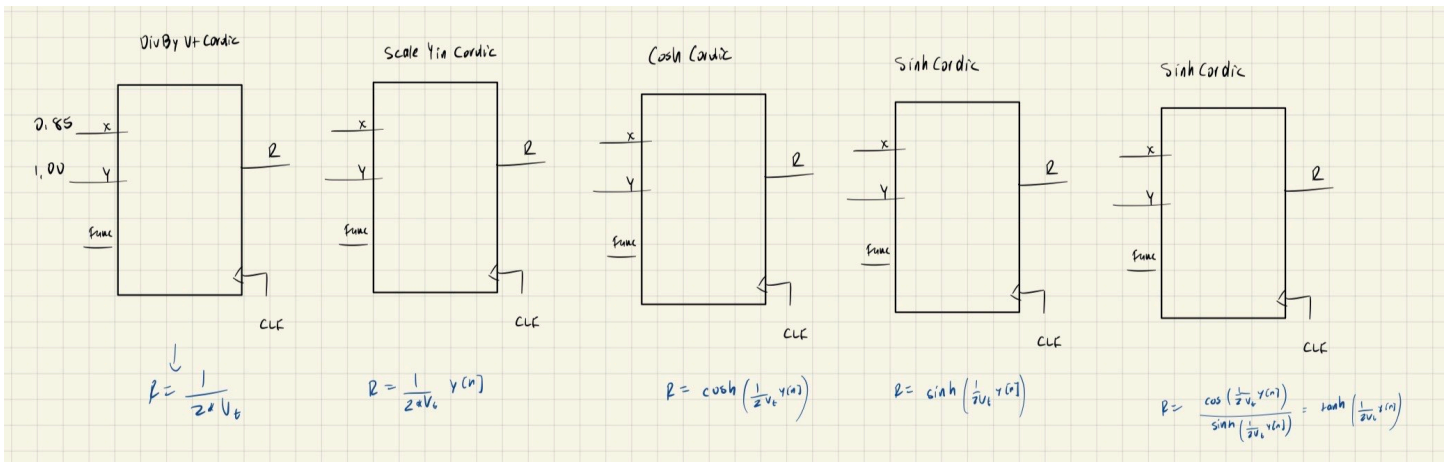


Figure 3: W block digital diagram

It is important to note that the finished design uses Q1.14 signed fixed point numbers (or Q2.14 depending on the naming) convention. This means that we have two bits for a signed integer part, and 14 bits for the fractional part. This was done because the CORDIC was rigorously tested in this configuration, but this severely limits range of intermediate values we can represent. Ideally, we would be able to represent the voltage range of original TB-303, which is most on the order of 10s of voltages.

Additionally, the CORDIC was modified so that multiple instantiations of it are possible, and was made fully concurrent to simplify prototyping. This will severely limit speed, but since our timing requirement is relatively large, we are okay with this trade off.

The WBlock was implemented in the wblock directory, specifically in WBlock.vhd, and a test bench confirmed it to be working with a few simple values.

The filter as a whole can be represented using the following diagram.

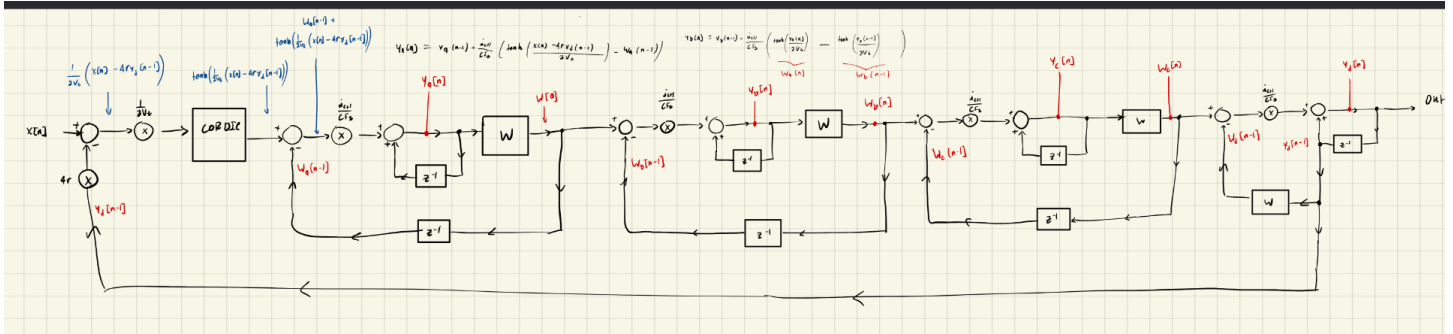


Figure 4: Moog filter diagram

From this, we see that the filter can be naturally broken up into roughly 5 stages:

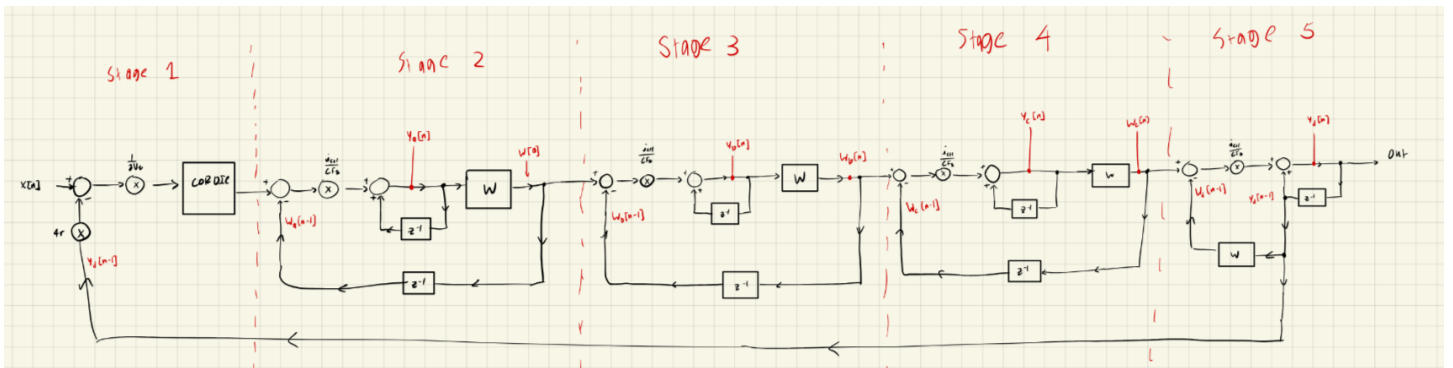


Figure 5: Moog filter diagram (in stages)

We see that the middle 3 stages are identical, and the 5th stage is quite similar to the others, although it does not have have a W block at its output like the other stages. We can thus create the following filter stage, to be reused in stages 2 - 5, with some modification.

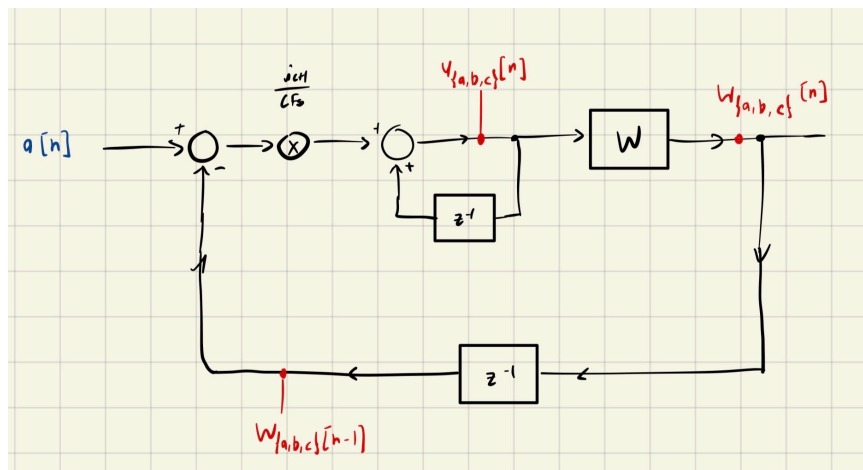


Figure 6: Moog filter stage

Here, we mark the input to the stage as some arbitrary $a[n]$, but concretely, this input is

- $\tanh(\frac{1}{2V_t}(x[n] - 4ry_d[n - 1]))$ for stage 2, where V_t is the thermal voltage of a transistor, $x[n]$ is the filter

input (the output of NCO), R is the simulated resistance value from the analog Moog filter, and C is the simulated capacitance value from the analog filter.

- $W_a[n]$ for Stage 3
- $W_b[n]$ for Stage 4
- $W_c[n]$ for Stage 5

Also note that although Stage 5 was drawn differently in the diagram, it is functionally equivalent to putting a W block after the $y_d[n]$ output, and putting a delay element in between the final W block and the first adder of the stage. The WBlock at the end of this stage will calculate $\tanh(\frac{y_d[n]}{2V_t})$, which becomes $\tanh(\frac{y_d[n-1]}{2V_t})$ after going through a delay element. We must note however, that the final output of the filter is y_d , and not the output of the final WBlock. For this reason, each stage was made to output both $y_{a,b,c,d}[n]$ respectively, as well as the output of its final W Block.

The filter stage was implemented in the `moogfilterstage` directory, specifically in `MoogFilterStage.vhd`. Its functionality was tested against a few input values, and it was shown to produce the expected values.

Finally, the filter stages were wired together in the `moogfilter` directory (in the `MoogFilter.vhd` file). ## Results

The NCO was shown to accurately produce the required notes in the bass clef.

The WBlock and `MoogFilterStage` were tested against a few values, and were found to produce the expected outputs.

Unfortunately, the output of `MoogFilter` oscillated between a valid value and an undefined value. I suspect this is a result of $y_d[n-1]$ being fed into the input of the first stage. Despite adding a synchronous reset signal and defaulting all clocked signals to zero, the issue was not fixed.

Please see the `screenshots` directory for images of the waveforms (including their fixed point representations). Additionally, please see the structure and location of the `txt` and `vcd` below.

Source Code structure

Each block has its own directory and Makefile. Each directory can be compiled using The main directories are

- `cordic` - used for the simulation (tested rigorously with OSVVM).
 - Main files:
 - * `CordicStage.vhd` - a single stage of the cordic
 - * `Cordic.vhd` - 16 stage Q1.14 cordic implementation
 - * `Cordic_TB.vhd` - OSVVM test bench
 - Artifacts `cordic/cordic_sim.log` - Cordic OSVVM verification for all valid functions and inputs
- `nco` - used for the NCO simulation
 - Main files:
 - * `NCO.vhd` - 16-bit numerically controlled oscillator outputting square and sawtooth waves
 - * `NCO_TB.vhd` - Test bench that sweeps over frequencies in the bass clef
 - Artifacts: `nco/nco_sim.txt` - runtime output (what freq/wave is being tested) `nco/sawtooth.txt` - sawtooth wave amplitudes over the frequency range `nco/square.txt` - square wave amplitudes over the frequency range `nco/verify_nco.txt` - harmonic analysis output `nco/work/NCO_TB.vcd` - value change dump file for test bench
- `wblock` - used for the wblock simulation (a part of the filter implementation)
 - Main files:

- * WBlock.vhd - A block that computes $\tanh(1/2Vt*y[n])$
- * WBlock_TB.vhd - Test bench that checks a few inputs for validity
- Artifacts:
 - * wblock/work/wblock_tb.vcd - value change dump file
- moogfilterstage - used for the simulation of a single stage of the moog ladder filter
 - Main files:
 - * MoogFilterStage.vhd - implementation of a single filter stage
 - * MoogFilterStage_TB.vhd - test bench that inputs a few values
 - Artifacts:
 - * moogfilterstage/work/moogfilterstaeg_tb.vcd - value change dump file
- moogfilter - used for the top level moog filter simulation
 - Main files:
 - * MoogFilterStage.vhd - implementation of a single filter stage
 - * MoogFilterStage_TB.vhd - test bench that inputs a few values
 - Artifacts:
 - * moogfilterstage/work/moogfilterstaeg_tb.vcd - value change dump file

Each simulation generates a VCD file that can be viewed in GTKwave (they are also saved as artifacts in github actions). The NCO and CORDIC test benches also generate text output files related to their analysis. The CORDIC simulation shows the expected and actual values, while NCO simulation outputs the amplitude output at each time step, as well as a file that shows the harmonics for each frequency.

Entire project structure

```
./
├── Dockerfile # dockerfile used for reproducible github build (doesn't work currently)
├── Makefile   # top level makefile (doesn't work currently)
├── README.md
├── TODO.md
├── build_osvvm.sh # osvvm build script
├── cordic/      # cordic directory. Note that this includes many versions of the cordic
│               # also note that these versions are BROKEN (you can't instantiate multiple
│               # of them because I have a shared variable bug in the CordicConstants package)
├── doc.md       # project documentation source code
├── doc.pdf      # project documentation PDF
├── filter/      # filter implementation. Includes all relevant files
│   ├── AddSub.vhd*           # 1-bit adder/subtractor
│   ├── AdderSubtractor.vhd*  # n-bit adder/subtractor
│   ├── AnsiEscape.vhd*       # AnsiEscape constants for colored output
│   ├── Cordic.vhd*           # 16-bit, Q1.14 (signed) fixed point cordic. Implemented internally
│   │                         # using Q3.18 signed fixed point numbers (22-bit). Modified to be
│   │                         # concurrent. Implemented using multiple stages
│   ├── CordicStage.vhd*      # A single Cordic stage
│   └── FixedLib.vhd*         # Real to fixed point standard_logic_vector conversion and vice-versa
├── Makefile      # filter make file
└── MoogFilter.vhd # Top level filter source file
```

```

├── MoogFilterStage.vhd      # A single stage of the moog ladder filter
├── MoogFilterStage_TB.vhd  # Test bench for single moog ladder filter stage
├── MoogFilter_TB.vhd       # test bench for top level moog ladder filter
├── WBlock.vhd              # W block source file (see above)
├── WBlock_TB.vhd           # W block test bench
├── make_filter.sh*         # shell script to build and run the filter
├── make_stage.sh*          # shell script to build and run the filter stage test bench
├── matlab/                 # matlab dev files
├── moogfilterstage_conf.gtkw
├── moogfilterstage_tb.vcd
├── out.txt
├── q1_14_filter.py*        # GTKwave filter to show fixed point numbers
├── work/
├── filter_diagram.jpg
├── gcc-12.5.0/             # GCC source code (for compiling GHDL 7.0.0 from source)
├──
├── nco/
├──   ├── INSTRUCTIONS.md    # build instructions
├──   ├── Makefile*          # makefile
├──   ├── nco.vhd*           # NCO source file
├──   ├── nco_tb.vhd*        # NCO test bench
├──   ├── out.txt            # output words from test bench
├──   ├── sawtooth.txt       # sawtooth only output words
├──   ├── sawtooth_freq_domain/ # frequency domain plots
├──   ├── sawtooth_time_domain/ # time domain plots
├──   ├── square.txt         # square wave only output words
├──   ├── square_freq_domain/ # frequency domain plots
├──   ├── square_time_domain/ # time domain plots
├──   ├── time_util.vhd      # ???
├──   ├── verify_nco.py*     # iterations of python script used to verify NCO
├──   ├── verify_nco.py.copy*
├──   ├── verify_nco_1.py
├──   ├── verify_nco_2.py
├──   └── verify_nco_3.py
├── sub.md                  # submission doc
├── w_block.jpg
└── w_block_digital.jpg

```

Building

The entire build succeeds in the Github actions page. Alternatively one could do

```
$ docker buildx build --target export --output . .
```

provided they have docker and docker build-x installed.

Also you could just run `make` run in each directory even with an older version of ghdl-gcc or ghdl-llvm (I tested it locally with GHDL 5.1.1).

Results and Reflection

There are many steps needed to make this a much better project that is more similar to the analog TB-303, more hardware efficient, and easier to understand.

Firstly, the bug in the implemented filter should be fixed, so that the filter as a whole is functional. The individual parts should then be wired together, and the design as a whole should be tested on commercially available simulators that can also be used to synthesize designs, such as Xilinx Vivado or Xilinx ISE.

There are also too many instances of the Cordic, which is unnecessary for simple tasks such as addition, multiplication, or division. Dedicated dividers and multipliers would greatly reduce the gate count and complexity of this project.

A more ideal filter would be derived using numerical analysis of an actual diode ladder filter instead of a BJT based ladder filter. Although the two sound similar, the TB-303's filter's dynamics are altered by its use of diodes. This would complicate the design however, since there is much less information available on diode ladder filters.

The design should be synthesized for an FPGA, and the hardware for a usable user interface should be implemented. This would include keys, a DAC, and a speaker. It would also be possible to implement my own Delta-Sigma DAC for this.

[1] https://dafx.de/paper-archive/2004/P_061.PDF